

Milestone 1

Group Members-

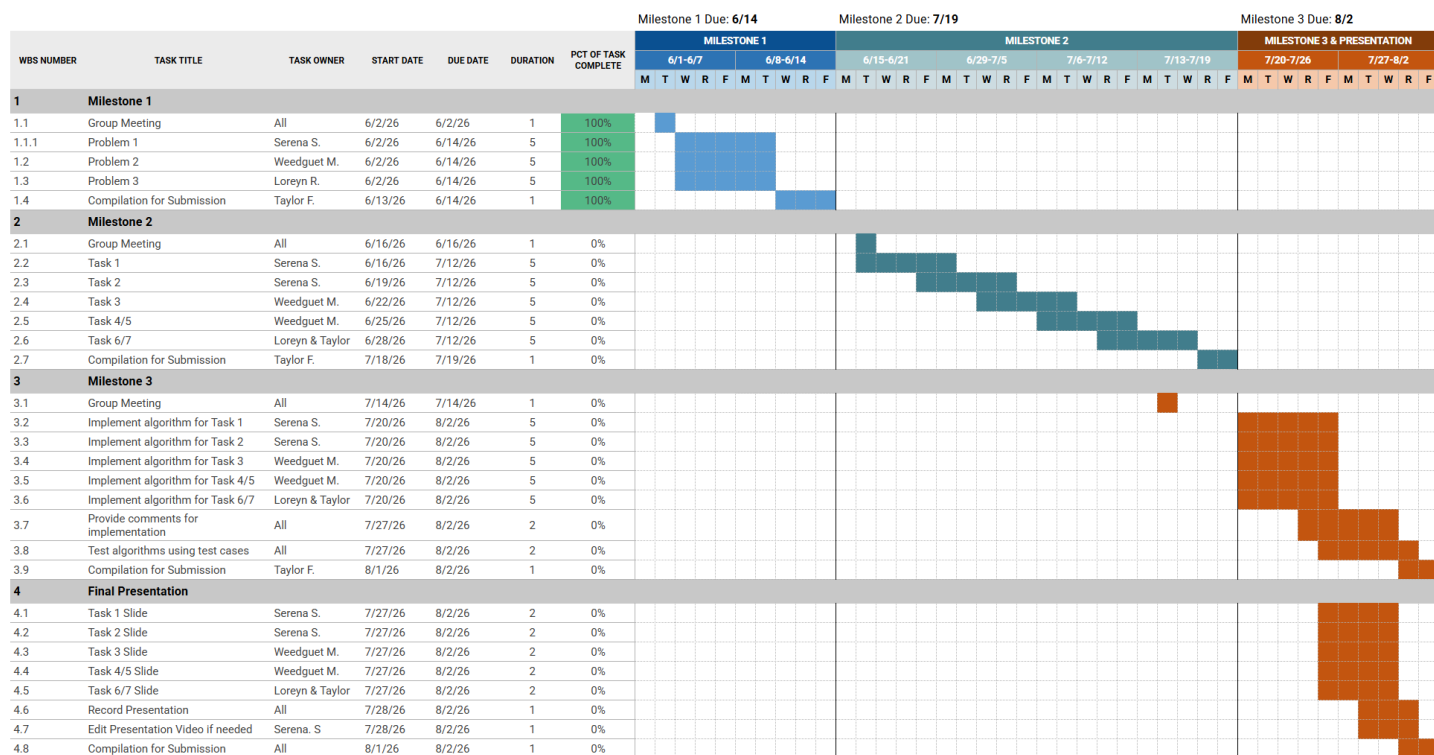
- Taylor Filson (tyler.filson@ufl.edu, GitHub: TFilson)
- Weedguet Mildort (weedguet.mildort@ufl.edu, GitHub: weedguetmildort)
- Loreyn Reid (L.reid1@ufl.edu, GitHub: loreyn626)
- Serena Seymour (serenaseymour@ufl.edu, GitHub: serenaseymour)

Member Roles-

- Taylor Filson: Development, Time Keeper
- Weedguet Mildort: Development
- Loreyn Reid: Development
- Serena Seymour: Development

Communication Method- Discord

Project Gantt Chart-



GitHub Repo- <https://github.com/serenaseymour/COP4533-Group-1-Summer-2026>

Solutions-

Problem Statement 1:

$$A = \begin{matrix} & \begin{matrix} \text{Day 1} & \text{Day 2} & \text{Day 3} & \text{Day 4} & \text{Day 5} \end{matrix} \\ \begin{matrix} \text{Stock 1} \\ \text{Stock 2} \\ \text{Stock 3} \\ \text{Stock 4} \end{matrix} & \begin{bmatrix} 12 & 1 & 5 & 3 & 16 \\ 4 & 4 & 13 & 4 & 9 \\ 6 & 8 & 6 & 1 & 2 \\ 14 & 3 & 4 & 8 & 10 \end{bmatrix} \end{matrix}$$

Stock 1				
Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome
1	2	1-12	-11	Loss
1	3	5-12	-7	Loss
1	4	3-12	-9	Loss
1	5	16-12	4	Good
2	3	5-1	4	Good
2	4	3-1	2	Good
2	5	16-1	15	Best
3	4	3-5	-2	Loss
3	5	16-5	11	Good
4	5	16-3	13	Good

Stock 2				
Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome
1	2	4-4	0	Even
1	3	13-4	9	Best
1	4	4-4	0	Even
1	5	9-4	5	Good
2	3	13-4	9	Best
2	4	4-4	0	Even
2	5	9-4	5	Good
3	4	13-4	9	Best
3	5	13-9	5	Good
4	5	4-9	-5	Loss

Stock 3				
Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome
1	2	8-6	2	Best
1	3	6-6	0	Even
1	4	1-6	-5	Loss
1	5	2-6	-4	Loss
2	3	6-8	-2	Loss
2	4	1-8	-7	Loss
2	5	2-8	-6	Loss
3	4	1-6	-5	Loss
3	5	2-6	-4	Loss
4	5	2-1	-1	Loss

Stock 4				
Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome
1	2	3-14	-11	Loss
1	3	4-14	-10	Loss
1	4	8-14	-6	Loss
1	5	10-14	-4	Loss
2	3	4-3	1	Good
2	4	8-3	5	Good
2	5	10-3	7	Best
3	4	8-4	4	Good
3	5	10-4	6	Good
4	5	10-8	2	Good

Stock 1 Best: **15**

Stock 2 Best: 9

Stock 3 Best: 2

Stock 4 Best: 7

Max Profit= 15
(1, 2, 5, 15)

Stock 1, bought on day 2 (1), sold on day 5 (16), profit= 15

Problem Statement 2:

$$A = \begin{matrix} & \begin{matrix} \text{Day 1} & \text{Day 2} & \text{Day 3} & \text{Day 4} & \text{Day 5} \end{matrix} \\ \begin{matrix} \text{Stock 1} \\ \text{Stock 2} \\ \text{Stock 3} \\ \text{Stock 4} \end{matrix} & \begin{bmatrix} 25 & 30 & 15 & 40 & 50 \\ 10 & 20 & 30 & 25 & 5 \\ 30 & 45 & 35 & 10 & 15 \\ 5 & 50 & 35 & 25 & 45 \end{bmatrix} \end{matrix}$$

$$k = 3$$

Stock 1					Stock 2				
Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome	Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome
1	2	30-25	5	Gain	1	2	20-10	10	Gain
1	3	15-25	-10	Loss	1	3	30-10	20	Best
1	4	40-25	15	Gain	1	4	25-10	15	Gain
1	5	50-25	25	Gain	1	5	5-10	-5	Loss
2	3	15-30	-15	Loss	2	3	30-20	10	Gain
2	4	40-30	10	Gain	2	4	25-20	5	Gain
2	5	50-30	20	Gain	2	5	5-20	-15	Loss
3	4	40-15	25	Gain	3	4	25-30	-5	Loss
3	5	50-15	35	Best	3	5	5-30	-25	Loss
4	5	50-40	10	Gain	4	5	5-25	-20	Loss

Stock 3					Stock 4				
Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome	Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome
1	2	45-30	15	Best	1	2	50-5	45	Best
1	3	35-30	5	Gain	1	3	35-5	30	Gain
1	4	10-30	-20	Loss	1	4	25-5	20	Gain
1	5	15-30	-15	Loss	1	5	45-5	40	Gain
2	3	35-45	-10	Loss	2	3	35-50	-15	Loss
2	4	10-45	-35	Loss	2	4	25-50	-25	Loss
2	5	15-45	-30	Loss	2	5	45-50	-5	Loss
3	4	10-35	-25	Loss	3	4	25-35	-10	Loss
3	5	15-35	-20	Loss	3	5	45-35	10	Gain
4	5	15-10	5	Gain	4	5	45-25	20	Gain

All the best profits:

Stock 1 Best: 35

Stock 2 Best: 20

Stock 3 Best: 15

Stock 4 Best: 45

The top two best profits are 35 and 45, and they are acquired between day 1 to day 2, and between day 3 to day 5, which leaves day 2 and day 3 available to take another transaction.

All the results between day 2 and 3:

Stock 1 Best: -15

Stock 2 Best: 10

Stock 3 Best: -10

Stock 4 Best: -15

The best result between day 2 and day 3 is stock 2.

A sequence of at most k transactions, each involving the purchase and sale of a single stock, that yields the maximum profit is: [(4, 1, 2),(2, 2, 3),(1, 3, 5)]

Stock 4, bought on day 1 (5), sold on day 2 (50), profit= 45

Stock 2, bought on day 2 (20), sold on day 3 (30), profit= 10

Stock 1, bought on day 3 (15), sold on day 5 (50), profit= 35

Hence the total maximum possible profit is 90. To reach the maximum possible profit, the first step was to identify the two best transactions, then determine the best third transaction that satisfied all constraints.

Problem Statement 3:

$$A = \begin{matrix} & \begin{matrix} \text{Day 1} & \text{Day 2} & \text{Day 3} & \text{Day 4} & \text{Day 5} & \text{Day 6} & \text{Day 7} \end{matrix} \\ \begin{matrix} \text{Stock 1} \\ \text{Stock 2} \\ \text{Stock 3} \\ \text{Stock 4} \\ \text{Stock 5} \end{matrix} & \begin{bmatrix} 7 & 1 & 5 & 3 & 6 & 8 & 9 \\ 2 & 4 & 3 & 7 & 9 & 1 & 8 \\ 5 & 8 & 9 & 1 & 2 & 3 & 10 \\ 9 & 3 & 4 & 8 & 7 & 4 & 1 \\ 3 & 1 & 5 & 8 & 9 & 6 & 4 \end{bmatrix} \end{matrix} \quad c = 2$$

Based on our current example of 7 consecutive days and the 2 day trading restriction, the max number of full transactions (buy and sell) is 2. To be able to make a second full transaction, the sell day of transaction 1 can be no later than day 3. The purchase date of transaction 2 can be no later than day 6.

T1: Transaction 1 Results

Note: Eligibility to buy again is based on the statement above. If the sell day of transaction 1 is after day 3, then for calculation purposes, it is marked not eligible to buy again. There would not be enough time to make a second full transaction given the trading restriction.

Best E refers to the highest profit of those eligible to buy again. Best O is the overall highest profit.

Stock 1					
Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome	Eligible to Buy Again (E)
1	2	1-2	-6	Loss	Y - On Days 5 or 6
1	3	2-3	-2	Loss	Y - On Day 6
1	4	3-4	-4	Loss	N
1	5	4-5	-1	Loss	N
1	6	5-6	1	Gain	N
1	7	6-7	2	Gain	N
2	3	1-2	4	Gain (Best E)	Y - On Day 6
2	4	2-3	2	Gain	N
2	5	3-4	5	Gain	N
2	6	4-5	7	Gain	N
2	7	5-6	8	Gain (Best O)	N
3	4	1-2	-2	Loss	N
3	5	2-3	1	Gain	N
3	6	3-4	3	Gain	N
3	7	4-5	4	Gain	N
4	5	1-2	3	Gain	N
4	6	2-3	5	Gain	N
4	7	3-4	6	Gain	N
5	6	1-2	2	Gain	N
5	7	2-3	3	Gain	N
6	7	1-2	1	Gain	N

COP4533
Summer 2026
Group 1

Stock 2					
Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome	Eligible to Buy Again (E)
1	2	4-2	2	Gain (Best E)	Y - On Days 5 or 6
1	3	3-2	1	Gain	Y - On Day 6
1	4	7-2	5	Gain	N
1	5	9-2	7	Gain (Best O)	N
1	6	1-2	-1	Loss	N
1	7	8-2	6	Gain	N
2	3	3-4	-1	Loss	Y - On Day 6
2	4	7-4	3	Gain	N
2	5	9-4	5	Gain	N
2	6	1-4	-3	Loss	N
2	7	8-4	4	Gain	N
3	4	7-3	4	Gain	N
3	5	9-3	6	Gain	N
3	6	1-3	-2	Loss	N
3	7	8-3	5	Gain	N
4	5	9-7	2	Gain	N
4	6	1-7	-6	Loss	N
4	7	8-7	1	Gain	N
5	6	1-9	-8	Loss	N
5	7	8-9	-1	Loss	N
6	7	8-1	7	Gain (Best O)	N

Stock 3					
Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome	Eligible to Buy Again (E)
1	2	8-5	3	Gain	Y - On Days 5 or 6
1	3	9-5	4	Gain (Best E)	Y - On Day 6
1	4	1-5	-4	Loss	N
1	5	2-5	-3	Loss	N
1	6	3-5	-2	Loss	N
1	7	10-5	5	Gain	N
2	3	9-8	1	Gain	Y - On Day 6
2	4	1-8	-7	Loss	N
2	5	2-8	-6	Loss	N
2	6	3-8	-5	Loss	N
2	7	10-8	2	Gain	N
3	4	1-9	-8	Loss	N
3	5	2-9	-7	Loss	N
3	6	3-9	-6	Loss	N
3	7	10-9	1	Gain	N
4	5	2-1	1	Gain	N
4	6	3-1	2	Gain	N
4	7	10-1	9	Gain (Best O)	N
5	6	3-2	1	Gain	N
5	7	10-2	8	Gain	N
6	7	10-3	7	Gain	N

COP4533
Summer 2026
Group 1

Stock 4					
Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome	Eligible to Buy Again (E)
1	2	3-9	-6	Loss	Y - On Days 5 or 6
1	3	4-9	-5	Loss	Y - On Day 6
1	4	8-9	-1	Loss	N
1	5	7-9	-2	Loss	N
1	6	4-9	-5	Loss	N
1	7	1-9	-8	Loss	N
2	3	4-3	1	Gain (Best E)	Y - On Day 6
2	4	8-3	5	Gain (Best O)	N
2	5	7-3	4	Gain	N
2	6	4-3	1	Gain	N
2	7	1-3	-2	Loss	N
3	4	8-4	4	Gain	N
3	5	7-4	3	Gain	N
3	6	4-4	0	Gain	N
3	7	1-4	-3	Loss	N
4	5	7-8	-1	Loss	N
4	6	4-8	-4	Loss	N
4	7	1-8	-7	Loss	N
5	6	4-7	-3	Loss	N
5	7	1-7	-6	Loss	N
6	7	1-4	-3	Loss	N

Stock 5					
Buy Day (j1)	Sell Day (j2)	(j2)-(j1)	Profit	Outcome	Eligible to Buy Again (E)
1	2	1-3	-2	Loss	Y - On Days 5 or 6
1	3	5-3	2	Gain	Y - On Day 6
1	4	8-3	5	Gain	N
1	5	9-3	6	Gain	N
1	6	6-3	3	Gain	N
1	7	4-3	1	Gain	N
2	3	5-1	4	Gain (Best E)	Y - On Day 6
2	4	8-1	7	Gain	N
2	5	9-1	8	Gain (Best O)	N
2	6	6-1	5	Gain	N
2	7	4-1	3	Gain	N
3	4	8-5	3	Gain	N
3	5	9-5	4	Gain	N
3	6	6-5	1	Gain	N
3	7	4-5	-1	Loss	N
4	5	9-8	1	Gain	N
4	6	6-8	-2	Loss	N
4	7	4-8	-4	Loss	N
5	6	6-9	-3	Loss	N
5	7	4-9	-5	Loss	N
6	7	4-6	-2	Loss	N

T2: Transaction 2 Results & Final Profits

For each transaction that was eligible for a second transaction, we compute the total profit of each combination of T1 and T2 to find the max profit.

Note: The Max Profit takes into account single transactions from the results above and the total profits from the 2nd transaction results below.

Transaction 2 (Stock 1 Transaction 1)						
Stock	Buy Day (j1)	Sell Day (j2)	Profit	Total Profit, T1: j1 = 1, j2 = 2, (-6 + profit)	Total Profit T1: j1 = 1, j2 = 3, (-2 + profit)	Total Profit T1: j1 = 2, j3 = 3, (4 + profit)
1	5	6	2	-4		
1	5	7	3	-3		
1	6	7	1	-5	-1	5
2	5	6	-8	-14		
2	5	7	-1	-7		
2	6	7	7	1	5	11
3	5	6	1	-5		
3	5	7	8	2		
3	6	7	7	1	5	11
4	5	6	-3	-9		
4	5	7	-6	-12		
4	6	7	-3	-9	-5	1
5	5	6	-3	-9		
5	5	7	-5	-11		
5	6	7	-2	-8	-4	2

Stock 1:

Max Profit = 11

[(1, 2, 3), (2,6,7)]

[(1, 2, 3), (3,6,7)]

Transaction 2 (Stock 2 Transaction 1)						
Stock	Buy Day (j1)	Sell Day (j2)	Profit	Total Profit, T1: j1 = 1, j2 = 2, (2 + profit)	Total Profit T1: j1 = 1, j2 = 3, (1 + profit)	Total Profit T1: j1 = 2, j3 = 3, (-1 + profit)
1	5	6	2	4		
1	5	7	3	5		
1	6	7	1	3	2	0
2	5	6	-8	-6		
2	5	7	-1	1		
2	6	7	7	9	8	6
3	5	6	1	3		
3	5	7	8	10		
3	6	7	7	9	8	6
4	5	6	-3	-1		
4	5	7	-6	-4		
4	6	7	-3	-1	-2	-4
5	5	6	-3	-1		
5	5	7	-5	-3		
5	6	7	-2	0	-1	-3

Stock 2:

Max Profit = 10

[(2, 1, 2), (3,5,7)]

Transaction 2 (Stock 3 Transaction 1)						
Stock	Buy Day (j1)	Sell Day (j2)	Profit	Total Profit, T1: j1 = 1, j2 = 2, (3 + profit)	Total Profit T1: j1 = 1, j2 = 3, (4 + profit)	Total Profit T1: j1 = 2, j3 = 3, (1 + profit)
1	5	6	2	5		
1	5	7	3	6		
1	6	7	1	4	5	2
2	5	6	-8	-5		
2	5	7	-1	2		
2	6	7	7	10	11	8
3	5	6	1	4		
3	5	7	8	11		
3	6	7	7	10	11	8
4	5	6	-3	0		
4	5	7	-6	-3		
4	6	7	-3	0	1	-2
5	5	6	-3	0		
5	5	7	-5	-2		
5	6	7	-2	1	2	-1

Stock 3:

Max Profit = 11

[(3, 1, 2), (3,5,7)]

[(3, 1, 3), (2,6,7)]

[(1, 1, 3), (3,6,7)]

Transaction 2 (Stock 4 Transaction 1)						
Stock	Buy Day (j1)	Sell Day (j2)	Profit	Total Profit, T1: j1 = 1, j2 = 2, (-6 + profit)	Total Profit T1: j1 = 1, j2 = 3, (-5 + profit)	Total Profit T1: j1 = 2, j3 = 3, (1 + profit)
1	5	6	2	-4		
1	5	7	3	-3		
1	6	7	1	-5	-4	2
2	5	6	-8	-14		
2	5	7	-1	-7		
2	6	7	7	1	2	8
3	5	6	1	-5		
3	5	7	8	2		
3	6	7	7	1	2	8
4	5	6	-3	-9		
4	5	7	-6	-12		
4	6	7	-3	-9	-8	-2
5	5	6	-3	-9		
5	5	7	-5	-11		
5	6	7	-2	-8	-7	-1

Stock 4:

Max Profit = 8

[(4, 2, 3), (2,6,7)]

[(4, 2, 3), (3,6,7)]

Transaction 2 (Stock 5 Transaction 1)						
Stock	Buy Day (j1)	Sell Day (j2)	Profit	Total Profit, T1: j1 = 1, j2 = 2, (-2 + profit)	Total Profit T1: j1 = 1, j2 = 3, (2 + profit)	Total Profit T1: j1 = 2, j3 = 3, (4 + profit)
1	5	6	2	0		
1	5	7	3	1		
1	6	7	1	-1	3	5
2	5	6	-8	-10		
2	5	7	-1	-3		
2	6	7	7	5	9	11
3	5	6	1	-1		
3	5	7	8	6		
3	6	7	7	5	9	11
4	5	6	-3	-5		
4	5	7	-6	-8		
4	6	7	-3	-5	-1	1
5	5	6	-3	-5		
5	5	7	-5	-7		
5	6	7	-2	-4	0	2

Stock 5:

Max Profit = 11

[(5, 2, 3), (2,6,7)]

[(5, 2, 3), (3,6,7)]

The overall max profit is 11. This could be obtained through multiple transactions. All are listed below.

[(1, 2, 3), (2,6,7)]

[(1, 2, 3), (3,6,7)]

[(3, 1, 2), (3,5,7)]

[(3, 1, 3), (2,6,7)]

[(1, 1, 3), (3,6,7)]

[(5, 2, 3), (2,6,7)]

[(5, 2, 3), (3,6,7)]

We only need to output one of these results:

Output: [(1, 2, 3), (2,6,7)]

Milestone 2

Programming Language: Python

Task 1: Brute force algorithm for Problem 1 in $O(m \cdot n^2)$

```
function BruteForce(A, m, n):  
    // A is 0-based 2D array of size m x n  
    max_profit = -1  
    best_stock = 0  
    best_buy = 0  
    best_sell = 0  
  
    for i = 1 to m:           // You go through every single stock one by one. m is the number of rows (stocks)  
        for j1 = 1 to n:      // For the current stock, you try every possible day as a buy day. j1 is the buy day, n is number of  
                               // columns (days)  
            for j2 = j1 + 1 to n: // For each buy day chosen, try every later day as a sell day, ensuring you only sell after buying  
                                   // (j1 < j2). This inner loop is what makes it  $O(n^2)$  per stock  
                profit = A[i-1][j2-1] - A[i-1][j1-1] // Calculate profit. You subtract 1 from indices because arrays in most  
                                                         // programming languages are 0-based. This gives sell price minus buy price  
                if profit > max_profit: // If this transaction is better than anything you've seen across all stocks and all pairs, the  
                                         // algorithm remembers it. Update the best profit and the exact stock/buy/sell days  
                    max_profit = profit  
                    best_stock = i  
                    best_buy = j1  
                    best_sell = j2  
  
    if max_profit > 0: // Final check. If you found any profitable transaction, return the best one. Otherwise return (0,0,0,0)  
        return (best_stock, best_buy, best_sell, max_profit)  
    else:  
        return (0, 0, 0, 0)
```

Time complexity: m stocks $\times$ n buy days $\times$ $\sim n$ sell days = $O(m \cdot n^2)$

Task 2: Greedy algorithm for Problem 1 in $O(m \cdot n)$

```
function Greedy(A, m, n):
    // A is 0-based 2D array of size m x n
    max_profit = -1
    best_stock = 0
    best_buy = 0
    best_sell = 0

    for i = 1 to m:           // Same as brute force, go through every stock one by one. m is the number of rows (stocks)
        if n < 2:             // If there are less than 2 days, you can't do a transaction. Skip this stock
            continue

        min_price = A[i-1][0] // Start with the price on day 1 as the minimum (best buy so far). Record that you would buy
                               // on day 1
        min_day = 1
        local_max_profit = 0
        local_buy = 1
        local_sell = 1

        for j = 2 to n:       // Scan every day starting from day 2. This is only one pass per stock, that's why it's O(n)
            current_profit = A[i-1][j-1] - min_price // If you sell today (day j), what profit would you get using the cheapest
                                                       // price seen before today?

            if current_profit > local_max_profit:     // If selling today gives a better profit than any previous sell opportunity
                                                       // on this stock, remember this profit and the days

                local_max_profit = current_profit
                local_buy = min_day
                local_sell = j

            if A[i-1][j-1] < min_price:             // If today's price is cheaper than the previous minimum, update the minimum price
                                                       // and the buy day. This prepares for future potential sell days

                min_price = A[i-1][j-1]
                min_day = j

        if local_max_profit > max_profit:           // After finishing all days for this stock, check if this stock's best transaction is
                                                       // better than the best you've seen from other stocks. If yes, update the global
                                                       // best

            max_profit = local_max_profit
            best_stock = i
            best_buy = local_buy
            best_sell = local_sell

    if max_profit > 0:
        return (best_stock, best_buy, best_sell, max_profit)
    else:
        return (0, 0, 0, 0)
```

Time complexity: $m \text{ stocks} \times n \text{ days} = O(m \cdot n)$

Task 3: Dynamic Algorithm for Problem 1 in $O(m \cdot n)$

To solve this problem dynamically in $O(m \cdot n)$ time we will keep a table that tracks the minimum priced day before each possible sell date. We will do so with 3 separate $n * m$ loops.

This solution is similar to the greedy approach though technically runs worse. This is an example of a problem that is best solved by a greedy approach (especially considering the space complexity that is cut out by not employing dynamic programming for this problem).

The recursive subproblem here is that in order to calculate the max profit, we first need to find the minimum priced day from the days before to buy each respective stock.

$M[i, j]$ stores the index of the minimum buy price before any given day for stock i on day j .

function dynamicOne(A, m, n):

STEP ONE:

Create a two-dimensional array M with dimensions $m \times n$, with each value initialized to infinity.

STEP TWO:

For each i to m //iterate through each stock (by index)

For each j (starting at the starting index + 1) to n //iterate through each day for each stock, skipping the first day of the time period.

if ($A[i, j-1] \leq A[i, M[i, j-1]]$): //if the value of the day prior is less than the minimum sell price up to that day,

$M[i, j] = j-1$ //set the minimum buy price before this day j to the index of the day prior

Else:

$M[i, j] = M[i, j-1]$ //Otherwise, retain the minimum buy price of the day before.

STEP THREE:

tempMax = 0 //pure max profit

tempBuy = 0 //index of when to buy

tempSell = 0 //index of when to sell

tempStock = 0 //index of which stock we're buying/selling

For each i to m:

 For each j (starting at the starting index + 1) to n

 maxProfitForDay = $A[i, j] - A[i, M[i, j]]$ //The max profit we can get by selling on any day is going to be the price today minus the price on the minimum day before this one for this stock.

 if maxProfitForDay > tempMax: //keeping track of max alongside each of the values that we want to return by the end.

 tempMax = maxProfitForDay

 tempBuy = M[i, j]

 tempSell = j

 tempStock = i

if tempMax < 0:

 return (0, 0, 0, 0)

else:

 return (tempStock, tempBuy, tempSell, tempMax)

This algorithm will find the maximum profit transaction in $O(n*m)$ time.

Task 5: Dynamic Programming Algorithm for Problem 2 in $O(m \cdot n \cdot k)$

```
function DynamicProgramming(A, m, n, k):
    // A is a 0-based 2D array of size m x n, where m is the number of stocks and n is the number of days
    // k is the maximum number of transactions allowed
    if n < 2 or k == 0:
        return []                                     // If there are fewer than 2 days or no transactions allowed,
                                                    // no profitable buy-sell transaction can be made

    create DP[0...k][1...n]                          // Create a 2D table named DP
    create Choice[0...k][1...n]                      // Create another 2D table named Choice

    // DP[t][d] represents the maximum profit possible using at most t transactions
    // considering only days 1 through d
    for d = 1 to n:
        DP[0][d] = 0                                // With 0 transactions, profit is always 0
        Choice[0][d] = None

    for t = 1 to k:
        DP[t][1] = 0                                // On day 1, we cannot sell yet, so profit is 0
        Choice[t][1] = None

    for t = 1 to k:                                  // Try using up to t transactions
        for i = 1 to m:
            best_buy_profit[i] = DP[t-1][1] - A[i-1][0]
            best_buy_day[i] = 1                      // Start by assuming stock i is bought on day 1
                                                    // best_value keeps track of the best previous profit
                                                    // minus the price paid to buy stock i

        for d = 2 to n:                              // Scan days from left to right
            DP[t][d] = DP[t][d-1]
            Choice[t][d] = ("skip", t, d-1)          // First assume we do not sell anything on day d
                                                    // This carries forward the best profit from yesterday

            for i = 1 to m:                            // Try selling each stock on day d
                current_profit = best_buy_profit[i] + A[i-1][d-1]
                                                    // If we sell stock i today, this gives the total profit:
                                                    // previous best profit before/at the buy day
                                                    // plus today's selling price
                                                    // minus the buying price already stored in best_value
                if current_profit > DP[t][d]:
                    DP[t][d] = current_profit
                    Choice[t][d] = ("sell", i, best_buy_day[i], d)
                                                    // If selling stock i today gives a better profit,
                                                    // remember the stock, buy day, and sell day

            for i = 1 to m:                            // Update the best possible buy option for future days
                possible_buy_profit = DP[t-1][d] - A[i-1][d-1]
                                                    // This represents buying stock i on day d
                                                    // after completing at most t-1 transactions by day d
                if possible_buy_profit > best_buy_profit[i]:
                    best_buy_profit[i] = possible_buy_profit
                    best_buy_day[i] = d              // If buying today gives a better future opportunity,
                                                    // update the best buy day for this stock

    if DP[k][n] <= 0:
        return []                                     // If no positive profit is possible, return empty sequence

    result = BuildSolution(Choice, k, n)             // Helper function
    return result
```

Time complexity: m stocks $\times$ n days $\times$ k transactions = $O(m \cdot n \cdot k)$

Task 5 helper function:

```
function BuildSolution(Choice, t, d):  
  
    if t == 0 or d <= 1:  
        return [] // No transactions left or no possible sell day  
  
    if Choice[t][d] == None:  
        return []  
  
    if Choice[t][d].type == "skip":  
        return BuildSolution(Choice, t, d-1) // If no transaction ends on day d,  
                                              // move back to the previous day  
  
    else:  
        stock = Choice[t][d].stock  
        buy_day = Choice[t][d].buy_day  
        sell_day = Choice[t][d].sell_day  
  
        previous_transactions = BuildSolution(Choice, t-1, buy_day)  
        // Since this transaction buys on buy_day,  
        // earlier transactions must finish on or before buy_day.  
        // This matches the non-overlapping rule.  
  
        return previous_transactions + [(stock, buy_day, sell_day)]
```

Task 6: Dynamic Programming Algorithm for Problem 3 – $O(m \cdot n^2)$

Let's keep track of the total max profit possible by the end of day, d . To do this, we will focus on two cases. Case 1 is to do nothing on day, d . Case 2 is to sell on day d . If we choose to sell, then we will determine the best day to buy considering the max profit that can be obtained by combining any previous transactions. It is important to remember that if there is a previous transaction, its sell date must be on or before day $j-c-1$ (j is the current buy date we are considering).

To solve this problem dynamically in $O(m \cdot n^2)$, we used nested loops. First, we iterate through n days, then we iterate through m stocks, and then we iterate through up to $n-1$ buy days.

Function `DynamicProgramming(A, m, n, c):`

STEP ONE:

Create an array `DP` with dimensions $1 \times n$ where n is the number of days. // stores the max profit up to day d

Create an array `Choice` with dimensions $1 \times n$ where n is the number of days. // stores the choices made during each day. This could be a "skip" day in which neither a purchase or sale is made or it could store the details of a transaction (stock, buy day, sell day).

STEP TWO:

if $n < 2$: // if there are less than 2 days, no transactions can be made

 return [] // cannot return a sequence if no full transactions are made

for $d = 1$ to n : // for each day

 if $d = 1$:

$DP[1] = 0$ // On day 1, if you buy and sell on the same day, the profit is zero

$Choice[1] = (\text{"skip"})$

 else:

$DP[d] = DP[d-1]$ // on day 2 or later, first case is that we do not sell

$Choice[d] = (\text{"skip"})$

 for $i = 1$ to m : // for each stock

 for $j = 1$ to $d-1$: //for each buy day that we could have bought the stock

 if $(j-c-1) \geq 1$: //ensures that we only check valid days

$current_total_profit = DP[j-c-1] + A[i][d] - A[i][j]$

 // test 2nd case which would be choosing to sell today.

$DP[j-c-1] = \text{max profit from previous transactions by the end of day } (j-c-1)$. This is to ensure that we are adhering to the waiting period.

$A[i][d] - A[i][j] = \text{the profit from the sale} = \text{selling price} - \text{buy price}.$

```
else:
```

```
current_total_profit = A[i][d] - A[i][j]
```

```
if current_total_profit > DP[d]:           // if the current_total _profit is greater
```

than the previously stored profit of DP[d]

```
DP[d] = current_total_profit // update the profit amount
```

```
Choice[d] = (i, j, d)           // store the choice
```

```
result = BuildSolution(Choice, n, c )           // return the sequences that yield the max profit
```

```
return result
```

STEP 3:

```
function BuildSolution(Choice, d, c):
```

```
if d <= 1:
```

```
return []
```

```
if Choice[d].type == "skip"
```

```
return BuildSolution(Choice, d-1) // returns the choice from the previous
day since there would not be a sequence of a transaction on a skip day
```

else:

```
stock = Choice[d].stock           // assign variables to the values of Choice[d]
```

```
buy_day = Choice[d].buy_day
```

```
sell_day = Choice[d].sell_day
```

```
previous_transactions = BuildSolution(Choice, buy_day-c-1, c) // pull the
sequence from the previous transaction if there is one
```

```
return previous_transactions + [(stock, buy_day, sell_day)]           // return the
                             previous sequences if they exist and the current sequence
```

Time Complexity: $m \text{ stocks} \times n \text{ days} \times n-1 \text{ days} = O(m \cdot n^2)$.

Milestone 3

Task 1: Brute force algorithm for Problem 1 in $O(m \cdot n^2)$

```
def brute_force(A, m, n):  
    # Task 1: Brute Force -  $O(m \cdot n^2)$   
  
    # A= array, size m*n (m= rows (stocks), n= columns (days))  
    max_profit = -1  
    best_stock = 0  
    best_buy = 0  
    best_sell = 0  
  
    for i in range(1, m + 1): # Go through every stock  
        for j1 in range(1, n + 1): # Try every possible buy day  
            for j2 in range(j1 + 1, n + 1): # Try every later sell day  
                profit = A[i - 1][j2 - 1] - A[i - 1][j1 - 1] # Calculate profit  
                if profit > max_profit: # Track the best transaction seen so far, and update  
                    max_profit = profit  
                    best_stock = i  
                    best_buy = j1  
                    best_sell = j2  
  
    if max_profit > 0: # Return the best valid positive profit transaction  
        return (best_stock, best_buy, best_sell, max_profit)  
    else:  
        return (0, 0, 0, 0)
```

task1_bruteforce.py

My solution uses three nested loops: one for each stock, one for every possible buy day (j_1), and one for every later sell day (j_2). For each valid pair, I calculate profit and track the maximum along with the corresponding stock and days.

- Edge case check for insufficient days
- `max_profit = -1` initialization to distinguish no-profit cases
- 1-based output indices with 0-based list access
- Simple integer variables for best transaction tracking

No major changes from Milestone 2 pseudocode.

Task 2: Greedy algorithm for Problem 1 in $O(m \cdot n)$

```
def greedy(A, m, n):  
    # Task 2: Greedy -  $O(m \cdot n)$   
  
    # A= array, size m*n (m= rows (stocks), n= columns (days))  
    max_profit = -1  
    best_stock = 0  
    best_buy = 0  
    best_sell = 0  
  
    for i in range(1, m + 1): # Go through every stock  
        if n < 2: # If less than 2 days, skip stock  
            continue  
  
        min_price = A[i - 1][0] # Start with day 1  
        min_day = 1  
        local_max_profit = 0  
        local_buy = 1  
        local_sell = 1  
  
        for j in range(2, n + 1): # Single pass over days starting from day 2  
            current_profit = A[i - 1][j - 1] - min_price # Profit if you sell today using best buy so far  
            if current_profit > local_max_profit: # Update local best if better  
                local_max_profit = current_profit  
                local_buy = min_day  
                local_sell = j  
  
            if A[i - 1][j - 1] < min_price: # Update minimum price and day if today is cheaper than previous min  
                min_price = A[i - 1][j - 1]  
                min_day = j  
  
        if local_max_profit > max_profit: # Compare and update this stock's best with global best  
            max_profit = local_max_profit  
            best_stock = i  
            best_buy = local_buy  
            best_sell = local_sell  
  
    if max_profit > 0:  
        return (best_stock, best_buy, best_sell, max_profit)  
    else:  
        return (0, 0, 0, 0)
```

task2_greedy.py

For each stock, I perform a single left-to-right pass while maintaining the minimum price seen so far and its day. At each potential sell day, I compute profit using the current min price and update the local best if better. After each stock, I compare it with the global best.

- Running min_price and min_day variables
- Single for-loop per stock for linear time
- Same -1 initialization and final check as brute force

Task 3: Dynamic Programming Solution for Problem 1 in $O(m*n)$ time

```
def dynamic(A, m, n):
    # Task 3: Dynamic Solution for Problem 1

    # Step 1 is to create our 2-dimensional M array and fill it with infinite placeholder
    # values
    M = [[float('inf') for _ in range(n)] for _ in range(m)]

    # For step 2, we do our dynamic tabulation, processing M for each possible [i,j] index in
    # the [m,n] matrix
    for i in range(m):
        # 1 to n since it's not possible to sell on the first day anyway.
        for j in range(1, n):
            # This first if statement was the sole logical change to this algorithm from
            # milestone 2.
            # It bypasses the indexOutOfBounds error that would occur were we to try and
            # access index infinity.
            # Logically however, it changes nothing since the left side A[i][j-1] would
            # always have been less than infinity anyway.
            if M[i][j-1] == float('inf'):
                M[i][j] = j-1
            elif A[i][j-1] < A[i][M[i][j-1]]:
                M[i][j] = j-1
            else:
                M[i][j] = M[i][j-1]

    # Finally step 3, where we follow a similar path to the brute force solution
    # yet rely on our previously solved sub-problem solutions stashed in M
    temp_max = 0
    for i in range(m):
        for j in range(1, n):
            daily_max_profit = A[i][j]-A[i][M[i][j]]
            if daily_max_profit > temp_max:
                temp_max = daily_max_profit
                temp_buy = M[i][j]
                temp_sell = j
                temp_stock = i
    if temp_max <= 0:
        return (0, 0, 0, 0)
    else:
        return (temp_stock+1, temp_buy+1, temp_sell+1, temp_max)
```

task3_dynamic.py

My solution follows the same 3 step process as outlined in milestone 2.

1. Build our M array and initialize each value to infinity
2. Calculate M for each i, j pair, building off of prior subproblems' solutions.
3. Go through each i, j pair again, keeping track of the max profit solution encountered, calculating max profit on each i, j by calling upon the solutions held in M.

Unit Testing for Tasks 1, 2, and 3

Running Test Cases for Tasks 1, 2, and 3

```
1. Milestone 1 Example      Brute: (1, 2, 5, 15) | Greedy: (1, 2, 5, 15) | Dynamic: (1, 2, 5, 15) | PASS
2. Empty                   Brute: (0, 0, 0, 0) | Greedy: (0, 0, 0, 0) | Dynamic: (0, 0, 0, 0) | PASS
3. Same Prices             Brute: (0, 0, 0, 0) | Greedy: (0, 0, 0, 0) | Dynamic: (0, 0, 0, 0) | PASS
4. All Decreasing          Brute: (0, 0, 0, 0) | Greedy: (0, 0, 0, 0) | Dynamic: (0, 0, 0, 0) | PASS
5. Single Day              Brute: (0, 0, 0, 0) | Greedy: (0, 0, 0, 0) | Dynamic: (0, 0, 0, 0) | PASS
6. One Stock, No Profit    Brute: (0, 0, 0, 0) | Greedy: (0, 0, 0, 0) | Dynamic: (0, 0, 0, 0) | PASS
7. Simple Increasing       Brute: (1, 1, 4, 7) | Greedy: (1, 1, 4, 7) | Dynamic: (1, 1, 4, 7) | PASS
8. Multiple Peaks         Brute: (1, 2, 7, 9) | Greedy: (1, 2, 7, 9) | Dynamic: (1, 2, 7, 9) | PASS
9. Multiple Stocks        Brute: (2, 1, 2, 8) | Greedy: (2, 1, 2, 8) | Dynamic: (2, 1, 2, 8) | PASS
10. Best at End            Brute: (1, 2, 6, 4) | Greedy: (1, 2, 6, 4) | Dynamic: (1, 2, 6, 4) | PASS
11. Best on Stock 3       Brute: (3, 1, 2, 99) | Greedy: (3, 1, 2, 99) | Dynamic: (3, 1, 2, 99) | PASS
12. All Zero              Brute: (0, 0, 0, 0) | Greedy: (0, 0, 0, 0) | Dynamic: (0, 0, 0, 0) | PASS
13. Negative Profit Only   Brute: (0, 0, 0, 0) | Greedy: (0, 0, 0, 0) | Dynamic: (0, 0, 0, 0) | PASS
14. Large Price Difference Brute: (1, 1, 2, 100000) | Greedy: (1, 1, 2, 100000) | Dynamic: (1, 1, 2, 100000) | PASS
15. m=1, n=1000          Brute: (1, 1, 100, 99) | Greedy: (1, 1, 100, 99) | Dynamic: (1, 1, 100, 99) | PASS
16. m=50, n=10           Brute: (1, 1, 10, 90) | Greedy: (1, 1, 10, 90) | Dynamic: (1, 1, 10, 90) | PASS
17. Random Fluctuations   Brute: (1, 2, 5, 8) | Greedy: (1, 2, 5, 8) | Dynamic: (1, 2, 5, 8) | PASS
18. Equal Max Multiple    Brute: (1, 1, 2, 9) | Greedy: (1, 1, 2, 9) | Dynamic: (1, 1, 2, 9) | PASS
19. Late Best Buy         Brute: (1, 4, 5, 85) | Greedy: (1, 4, 5, 85) | Dynamic: (1, 4, 5, 85) | PASS
20. Single Stock One Peak Early Brute: (1, 1, 2, 9) | Greedy: (1, 1, 2, 9) | Dynamic: (1, 1, 2, 9) | PASS
21. Buy on Last Day Impossible Brute: (1, 1, 2, 10) | Greedy: (1, 1, 2, 10) | Dynamic: (1, 1, 2, 10) | PASS
22. Zero Profit Transaction Only Brute: (1, 1, 4, 5) | Greedy: (1, 1, 4, 5) | Dynamic: (1, 1, 4, 5) | PASS
23. Prices with Duplicates Brute: (1, 2, 5, 5) | Greedy: (1, 2, 5, 5) | Dynamic: (1, 2, 5, 5) | PASS
24. All Stocks Peak on Same Day Brute: (3, 1, 2, 25) | Greedy: (3, 1, 2, 25) | Dynamic: (3, 1, 2, 25) | PASS
25. Minimal Profitable Transaction Brute: (1, 1, 2, 1) | Greedy: (1, 1, 2, 1) | Dynamic: (1, 1, 2, 1) | PASS
26. m=0 Empty Stocks      Brute: (0, 0, 0, 0) | Greedy: (0, 0, 0, 0) | Dynamic: (0, 0, 0, 0) | PASS
27. n=2 Minimal Case Profitable Brute: (1, 1, 2, 50) | Greedy: (1, 1, 2, 50) | Dynamic: (1, 1, 2, 50) | PASS
28. n=2 Minimal Case Loss Brute: (0, 0, 0, 0) | Greedy: (0, 0, 0, 0) | Dynamic: (0, 0, 0, 0) | PASS
29. Fluctuating with Multiple Good Options Brute: (1, 1, 4, 19) | Greedy: (1, 1, 4, 19) | Dynamic: (1, 1, 4, 19) | PASS
30. High Volatility Last Stock Best Brute: (3, 1, 2, 99) | Greedy: (3, 1, 2, 99) | Dynamic: (3, 1, 2, 99) | PASS

Results: 30/30 tests passed
```

task1&2_unittests.py

I created a separate test file that imports both functions and runs 30 test cases covering:

- The Milestone 1 example
- No-profit / edge cases
- Classic increasing patterns
- Multiple stocks with varying peaks
- Various other edge cases

Each test runs all three algorithm implementations and verifies consistency. Output shows pass/fail per case with total summary. This ensures thorough validation across inputs. I used lists for matrices, for-loops for iteration, and conditionals for updates/edges. All unit tests pass.

Analysis of Tasks 1, 2, and 3

Assumptions

- Input matrix is valid (m rows, n columns, non-negative ints)
- Any optimal transaction is acceptable if multiple exist

Limitations

- Task 1: $O(m \cdot n^2)$ time; for $m=n=1000$, $\sim 10^9$ operations, slow for Python
- Task 2: Greedy relies on problem property; does not generalize easily to k-transactions or cooldown

Comparative Analysis & Trade-offs

Correctness: All three produce identical outputs on all unit tests, matching Milestone 1's manual solution

Time Complexity:

- Brute Force: $O(m \cdot n^2)$; exhaustive
- Greedy: $O(m \cdot n)$; linear scan per stock
- Dynamic: $O(m \cdot n)$; 2 linear scans per stock-day

Space:

- Brute Force: $O(1)$
- Greedy: $O(1)$
- Dynamic: $O(n \cdot m)$; stores the solution for each stock-day pair

Readability/Maintainability: Brute force is simpler to verify. Greedy requires deeper insight but is more efficient. Dynamic is similar to brute force, but the array M is confusing until explained.

Trade-offs:

- Brute Force: Easy to implement/understand, poor scalability
- Greedy: Faster, demonstrates optimization via problem structure, but less general than dynamic programming
- Implementing both reinforced brute force verification before optimization
- Dynamic: Faster than brute force, but slower and less space efficient than Greedy.
- Implementing Dynamic following both Greedy and Brute Force showed how the three relate. Dynamic is faster than brute force, but has space complications. Yet if a greedy algorithm can optimally solve a problem, it's oftentimes better than the dynamic solution.

Analysis of Algorithms

Brute force explores the full search space, guaranteeing optimality through enumeration. Greedy exploits optimal substructure, achieving the same result with far fewer operations. Dynamic stores the redundant subproblems encountered in the Brute Force algorithm, saving time with the cost of space. All three correctly solve single-transaction max profit across stocks.

Lessons Learned

- Brute force is important for building intuition and testing efficient solutions
- Greedy algorithms succeed when local choices lead to global optimum
- Brute force solutions are often worse than greedy ones, but can solve many more problems
- Thorough testing via unit tests revealed edge cases I initially missed
- Separate files and clear comments improve collaboration and understanding
- Time complexity matters even for small n when scaling to constraints

Limitations

- Task 1 does not scale well to max $m/n=1000$

COP4533

Summer 2026

Group 1

- Task 2 cannot directly extend to Problems 2 and 3 with multiple transactions
- Task 3 requires space whilst the previous two do not.

Task 5: Dynamic Programming Solution for Problem 2 in $O(m \cdot n \cdot k)$ time

```
# Necessary imports
from collections import namedtuple

# Two record types stored in the Choice table, one per kind of decision
Skip = namedtuple("Skip", ["type", "t", "day"]) # carry yesterday's profit forward
Sell = namedtuple("Sell", ["type", "stock", "buy_day", "sell_day"]) # a transaction ends on this day

# Returns a list of (stock, buy_day, sell_day)
def dynamic_programming(A, m, n, k):
    # A is an array of size m * n (m rows (stocks), n columns (days))
    # k is maximum number of transactions allowed

    if n < 2 or k <= 0: # Fewer than 2 days or no transactions allowed, so no
        return [] # profitable buy-sell transaction can be made

    # DP[t][d] = maximum profit using at most t transactions over days 1 through d
    # Choice[t][d] = the decision that produced DP[t][d], used to rebuild the answer
    # Column 0 of each row is unused so the tables can be indexed by day directly
    DP = [[0] * (n + 1) for _ in range(k + 1)]
    Choice = [[None] * (n + 1) for _ in range(k + 1)]

    for d in range(1, n + 1): # Base case: with 0 transactions, profit is always 0
        DP[0][d] = 0
        Choice[0][d] = None

    for t in range(1, k + 1): # Base case: on day 1 we cannot sell yet, so profit is 0
        DP[t][1] = 0
        Choice[t][1] = None

    for t in range(1, k + 1): # Try using up to t transactions
        best_buy_profit = [0] * (m + 1)
        best_buy_day = [1] * (m + 1)

        for i in range(1, m + 1): # Start by assuming stock i is bought on day 1
            best_buy_profit[i] = DP[t - 1][1] - A[i - 1][0]
            best_buy_day[i] = 1

        for d in range(2, n + 1): # Scan days from left to right
            DP[t][d] = DP[t][d - 1]
            Choice[t][d] = Skip("skip", t, d - 1)

        for i in range(1, m + 1): # Try selling each stock on day d
            current_profit = best_buy_profit[i] + A[i - 1][d - 1]
            if current_profit > DP[t][d]: # Selling stock i today is better, so
                DP[t][d] = current_profit # remember the stock, buy day, and sell day
                Choice[t][d] = Sell("sell", i, best_buy_day[i], d)
```

Task5_dp.py

Our solution follows the general approach outlined in the Milestone 2 pseudocode, with adjustments for Python indexing and transaction reconstruction.

The algorithm uses 1-based indexing in the dynamic-programming and choice tables so that the table indices correspond directly to the stock and day numbers required by the problem statement. Additional logic was included to return an empty list when no strictly positive profit can be made.

```
        for i in range(1, m + 1): # Update the best buy option for future days
            possible_buy_profit = DP[t - 1][d] - A[i - 1][d - 1]
            if possible_buy_profit > best_buy_profit[i]: # Buying today gives a better
                best_buy_profit[i] = possible_buy_profit # future opportunity, so update
                best_buy_day[i] = d # the best buy day for this stock

    if DP[k][n] <= 0: # No positive profit is possible, so make no trades
        return []

    return build_solution(Choice, k, n)

# Walks the Choice table backwards from cell [t][d] to rebuild the solution
def build_solution(Choice, t, d):
    if t == 0: # No transactions left
        return []

    # Step back over days that end in no transaction
    while d > 1 and Choice[t][d] is not None and Choice[t][d].type == "skip":
        d -= 1

    if d <= 1 or Choice[t][d] is None: # No possible sell day, or nothing recorded
        return []

    stock = Choice[t][d].stock
    buy_day = Choice[t][d].buy_day
    sell_day = Choice[t][d].sell_day

    # This transaction buys on buy_day, so any earlier transactions must
    # finish on or before buy_day to match the non-overlapping rule
    previous_transactions = build_solution(Choice, t - 1, buy_day)
    return previous_transactions + [(stock, buy_day, sell_day)]
```

Task5_dp.py (continued)

As also outlined in the Milestone 2 pseudocode, there is a helper function to help build the solution towards the end to avoid clustering the main function logic.

Running Test Cases for Task 5

1. Spec Example k=3 (given) profit 17, expected 17	k=3	[(2, 1, 2), (1, 2, 3), (2, 3, 5)] PASS
2. Spec Example k=1 profit 7, expected 7	k=1	[(2, 1, 2)] PASS
3. Spec Example k=2 profit 13, expected 13	k=2	[(2, 1, 2), (2, 3, 5)] PASS
4. Problem 2 Example k=1 profit 45, expected 45	k=1	[(4, 1, 2)] PASS
5. Problem 2 Example k=2 profit 80, expected 80	k=2	[(4, 1, 2), (1, 3, 5)] PASS
6. Problem 2 Example k=3 (given) profit 90, expected 90	k=3	[(4, 1, 2), (2, 2, 3), (1, 3, 5)] PASS
7. Problem 2 Example k=4 profit 100, expected 100	k=4	[(4, 1, 2), (2, 2, 3), (1, 3, 4), (4, 4, 5)] PASS
8. Problem 2 Example k=5 Saturates profit 100, expected 100	k=5	[(4, 1, 2), (2, 2, 3), (1, 3, 4), (4, 4, 5)] PASS
9. Problem 1 Example k=1 profit 15, expected 15	k=1	[(1, 2, 5)] PASS
10. Problem 1 Example k=2 profit 22, expected 22	k=2	[(2, 1, 3), (1, 4, 5)] PASS
11. Problem 1 Example k=3 profit 26, expected 26	k=3	[(2, 1, 3), (4, 3, 4), (1, 4, 5)] PASS
12. Problem 1 Example k=4 profit 28, expected 28	k=4	[(3, 1, 2), (2, 2, 3), (4, 3, 4), (1, 4, 5)] PASS
13. Empty profit 0, expected 0	k=3	[] PASS
14. Same Prices profit 0, expected 0	k=2	[] PASS
15. All Decreasing profit 0, expected 0	k=2	[] PASS
16. Single Day profit 0, expected 0	k=3	[] PASS
17. One Stock, No Profit profit 0, expected 0	k=4	[] PASS
18. All Zero profit 0, expected 0	k=2	[] PASS
19. Negative Profit Only profit 0, expected 0	k=3	[] PASS
20. k=0 On Profitable Grid profit 0, expected 0	k=0	[] PASS
21. Negative k profit 0, expected 0	k=-1	[] PASS
22. k Larger Than Useful profit 10, expected 10	k=10	[(1, 1, 2), (1, 3, 4)] PASS
23. k Caps A Better Answer profit 7, expected 7	k=1	[(1, 1, 4)] PASS
24. Simple Increasing profit 7, expected 7	k=1	[(1, 1, 4)] PASS

```
25. Multiple Peaks          k=1 [(1, 2, 7)]
    profit 9, expected 9      PASS
26. Multiple Stocks         k=1 [(2, 1, 2)]
    profit 8, expected 8      PASS
27. Best at End             k=1 [(1, 2, 6)]
    profit 4, expected 4      PASS
28. Best on Stock 3         k=1 [(3, 1, 2)]
    profit 99, expected 99    PASS
29. Late Best Buy           k=1 [(1, 4, 5)]
    profit 85, expected 85    PASS
30. Large Price Difference   k=1 [(1, 1, 2)]
    profit 100000, expected 100000 PASS
31. Minimal Profitable Transaction k=1 [(1, 1, 2)]
    profit 1, expected 1      PASS
32. n=2 Minimal Case Loss   k=1 []
    profit 0, expected 0      PASS
33. Two Peaks One Stock     k=2 [(1, 1, 4), (1, 5, 6)]
    profit 31, expected 31    PASS
34. Equal Max Multiple      k=2 [(1, 1, 2), (1, 3, 4)]
    profit 18, expected 18    PASS
35. Sawtooth k=3            k=3 [(1, 1, 2), (1, 3, 4), (1, 5, 6)]
    profit 12, expected 12    PASS
36. Sawtooth Limited to k=2 k=2 [(1, 1, 2), (1, 3, 4)]
    profit 8, expected 8      PASS
37. Random Fluctuations     k=3 [(1, 2, 3), (1, 4, 5), (1, 6, 7)]
    profit 18, expected 18    PASS
38. Switch Stocks Between Trades k=2 [(1, 1, 2), (2, 3, 4)]
    profit 19, expected 19    PASS
39. Sell And Buy Same Day   k=2 [(1, 1, 2), (2, 2, 3)]
    profit 34, expected 34    PASS
40. High Volatility Last Stock Best k=2 [(3, 1, 2), (2, 2, 3)]
    profit 104, expected 104   PASS
41. Three Stocks Three Trades k=3 [(1, 1, 2), (2, 3, 4)]
    profit 13, expected 13    PASS
42. m=1, n=100 Increasing   k=1 [(1, 1, 100)]
    profit 99, expected 99    PASS
43. m=50, n=10             k=2 [(1, 1, 10)]
    profit 90, expected 90    PASS
```

Results: 43/43 tests passed

ALL TESTS PASSED

The test file imports the dynamic-programming implementation and conducts 43 test cases covering multiple possible edge cases. The test output displays whether each test case passes or fails, along with the transactions returned by the algorithm. All unit tests pass.

Description of the Algorithm Implementation

1. Initialize the dynamic-programming tables
 - a. Create a two-dimensional table, DP, where $DP[t][d]$ stores the maximum profit obtainable using at most (t) transactions during days 1 through (d).
 - b. Create a two-dimensional table, Choice, that records the decision used to produce each value in DP.
 - c. Initialize the base cases:
 - i. With zero transactions, the maximum profit is zero.
 - ii. On the first day, no transaction can be completed because a stock must be bought before it is sold.
2. Compute the maximum profit
 - a. Iterate through each transaction count from 1 through (k).
 - b. For every stock, maintain its best buying opportunity. The value stored for a stock represents:
 - c. $[DP[t-1][b] - A[i][b]]$ where (b) is a possible buying day.
 - d. Iterate through the days from left to right.
 - e. For each day, first carry forward the maximum profit from the previous day. This represents making no sale on the current day.
 - f. Try selling each stock on the current day. The potential profit is calculated as: $[best_buy_profit[i] + A[i][d]]$
 - g. If selling a stock produces a larger profit, update the corresponding DP entry and record the stock, buying day, and selling day in the Choice table.
 - h. After evaluating all possible sales for the current day, update the best buying opportunity for each stock. Updating the buying opportunities after evaluating sales ensures that the same transaction cannot buy and sell on the same day.
3. Reconstruct the selected transactions
 - a. Begin at $Choice[k][n]$, which represents the optimal result using at most (k) transactions over all (n) days.
 - b. Move backward through entries that indicate no transaction was completed on that day.
 - c. When a sale is found, record the stock number, buying day, and selling day.
 - d. Continue reconstructing the earlier transactions from the buying day of the current transaction.
 - e. Return the transactions in chronological order.

Limitations of the Algorithm

1. The $O(kmn)$ running time may still be expensive when (k), (m), and (n) are all large.
2. At the maximum stated values of $(k=m=n=1000)$, the algorithm may require approximately one billion stock-day evaluations, which is not practical in pure Python, a language closer to machine code may be more efficient.
3. The DP and Choice tables require $O(kn)$ auxiliary space.

Comparative Analysis

1. Time Complexity

The algorithm uses three main loops:

- Up to (k) transaction counts
- (n) days
- (m) stocks for each day

Therefore, the time complexity is: $O(kmn)$. This is more efficient than a brute force approach.

2. Space Complexity

The DP and Choice tables each contain $((k+1)(n+1))$ entries, requiring $O(kn)$ space. The arrays used to store the best buying profit and buying day for each stock require an additional $O(m)$ space. Therefore, the total space complexity is $O(kn+m)$.

Trade-Off Discussion

This dynamic programming algorithm requires more planning and additional memory compared with a simple brute-force solution. The algorithm must maintain both a profit table and a choice table so that the optimal transactions can be reconstructed. However, this additional space substantially reduces redundant calculations. Instead of repeatedly evaluating the same subproblems, the algorithm stores previously computed maximum profits and reuses them. The implementation also stores reconstruction information for every transaction count and day. This increases memory usage but allows the program to return the actual sequence of transactions rather than only the maximum profit. In other words, the algorithm uses significantly more memory than a brute force method, but also less time than that same brute force method.

Analysis of the algorithm

This dynamic programming algorithm improves efficiency by dividing the problem into smaller overlapping subproblems. The optimal profit for a particular day and transaction limit depends on optimal results that were already computed for earlier days and smaller transaction limits. The algorithm avoids repeatedly testing every possible combination of transactions. By maintaining the best buying opportunity for each stock, it also avoids an additional loop over all earlier buying days. This reduces the running time to $O(kmn)$.

Lessons Learned

1. Dynamic programming is useful when a problem contains overlapping subproblems and requires an optimal sequence of decisions.
2. The order of updates inside a dynamic-programming algorithm is important.
3. A choice table can be used to reconstruct the decisions that produced the optimal value.
4. Unit testing helps identify edge cases.
5. Pseudocode helps organize the recurrence, base cases, and reconstruction process before the algorithm is translated into code.
6. Complexity analysis is important because an algorithm can be mathematically correct while still being impractical for the largest possible inputs.

Limitations

The limitation of this algorithm like many dynamic programming algorithms is that testing very large matrices is difficult because the algorithm's running time, in this case, grows proportionally to (kmn) . Smaller and medium-sized cases can be tested efficiently, but inputs near the maximum constraints may require substantial execution time and memory.

Task 6: Dynamic Programming algorithm for Problem 3 in $O(m \cdot n^2)$

```
def dp(A, m, n, c):  
    # Task 6: Dynamic Programming -  $O(m \cdot n^2)$   
  
    DP = [0] * n # stores max profit up to day d  
    Choice = [None] * n # stores choices made during each day. either "skip" or (stock, buy_day, sell_day)  
  
    if n < 2: #if there are less than 2 days, no full transactions can be made  
        return []  
  
    for d in range(1, n+1): # for each day  
        if d == 1: # on day 1, the profit is zero, no profitable transactions can be made  
            DP[d-1] = 0  
            Choice[d-1] = "skip"  
  
        else:  
            DP[d-1] = DP[d-2] # on day 2 or later, option 1 is to skip day d and not sell  
            Choice[d-1] = "skip"  
  
            # option 2 is to sell on day d - check all stocks and previous buy days that adhere to the cooldown period  
            for i in range(1, m+1): # for each stock  
                for j in range(1, d): # for each buy day (exclude last day)  
                    if (j-c-1) >= 1: # ensure that we only check valid buy days that adhere to the cooldown period  
                        # max profit from previous eligible sell day + max profit from selling on day d  
                        # takes into account the cooldown period from the previous sell day  
                        current_total_profit = DP[j-c-2] + (A[i-1][d-1] - A[i-1][j-1])  
  
                        else:  
                            current_total_profit = A[i-1][d-1] - A[i-1][j-1]  
  
                        if current_total_profit > DP[d-1]: # checks if it is more profitable to skip day d or to sell on day d  
                            DP[d-1] = current_total_profit  
                            Choice[d-1] = (i, j, d)  
  
    if DP[n-1] == 0:  
        return [] # if the max profit is zero, return an empty list  
  
    result = BuildSolution(Choice, n, c) # return the sequences that yield the max profit  
    return result
```

```
def BuildSolution(Choice, d, c):  
    # Backtrack to find the sequences of transactions that lead to the max profit  
  
    if d <= 1: # no profit if there is one day or less  
        return []  
  
    if Choice[d-1] == "skip": # if the choice on day d is a skip day, return the choice from the previous day  
        return BuildSolution(Choice, d-1, c)  
  
    else:  
        stock, buy_day, sell_day = Choice[d-1]  
  
        previous_transactions = BuildSolution(Choice, buy_day - c - 1, c) # pull the sequence of the previous transaction if there is one  
        return previous_transactions + [(stock, buy_day, sell_day)] # return the previous sequences if they exist, and add the current transaction to the list
```

task6_dp.py

Our solution follows the same steps as outlined in the Milestone 2 Pseudocode. Slight adjustments were made for proper indexing. Also, code was added to return an empty set if the total profit is zero.

Unit Testing for Task 6

```
from task6_dp import dp

def tests():

    Example_Problem_3_1 =[
        [2,9,8,4,5,0,7],
        [6,7,3,9,1,0,8],
        [1,7,9,6,4,9,11],
        [7,8,3,1,8,5,2],
        [1,8,4,0,9,2,1]
    ]

    Example_Problem_3_2 =[
        [7,1,5,3,6,8,9],
        [2,4,3,7,9,1,8],
        [5,8,9,1,2,3,10],
        [9,3,4,8,7,4,1],
        [3,1,5,8,9,6,4]
    ]

    test_cases = [

        # Problem 3 Examples
        {"name": "Problem 3.1 example", "A": Example_Problem_3_1, "c": 2, "expected": [(3, 1, 3), (2, 6, 7)]},
        {"name": "Problem 3.2 example", "A": Example_Problem_3_2, "c": 2, "expected": [(1, 2, 3), (2, 6, 7)]},

        #No Profit Cases
        {"name": "Empty", "A": [], "c": 1, "expected": []},
        {"name": "Same Prices", "A": [[5, 5, 5, 5]], "c": 1, "expected": []},
        {"name": "All Decreasing", "A": [[10, 8, 7], [5, 4, 3]], "c": 1, "expected": []},
        {"name": "Single Day", "A": [[1], [2], [3]], "c": 1, "expected": []},
        {"name": "One Stock, No Profit", "A": [[5, 4, 3, 2, 1]], "c": 2, "expected": []},
        {"name": "All zeroes", "A": [[0, 0, 0], [0, 0, 0]], "c": 1, "expected": []},

        #Profitable/Increasing Cases
        {"name": "Simple Profit", "A": [[1, 2, 3, 4]], "c": 1, "expected": [(1, 1, 4)]},
        {"name": "Multiple Stocks", "A": [[1, 4], [2, 7]], "c": 1, "expected": [(2, 1, 2)]},
        {"name": "Multiple Transactions, Different Stocks", "A": [[1, 10, 3, 6, 1], [2, 4, 2, 1, 10]], "c": 1, "expected": [(1, 1, 2), (2, 4, 5)]},

        # Cooldown Cases
        {"name": "Large Cooldown", "A": [[1, 10, 1, 8, 9]], "c": 3, "expected": [(1, 1, 2)]},
        {"name": "Multiple Transactions with Cooldown", "A": [[1, 6, 1, 1, 8]], "c": 1, "expected": [(1, 1, 2), (1, 4, 5)]},

    ]

    print("Running Test Cases for Task 6\n ")
    passed = 0
    for idx, tc in enumerate(test_cases, 1): #idx = index, tc = test case
        A = tc["A"]
        m = len(A)
        n = len(A[0]) if m > 0 else 0
        c = tc["c"]

        dynamic = dp(A, m, n, c)

        expected = tc["expected"]
        success = dynamic == expected
        status = "PASS" if success else "FAIL"
        print(f"{idx:2d}. {tc['name']:30} c={c} | Dynamic: {dynamic} |{status}")
        if success:
            passed += 1

    print(f"\nResults: {passed}/{len(test_cases)} tests passed")

if __name__ == "__main__":
    tests()
```

Running Test Cases for Task 6

```
1. Problem 3.1 example      c=2 | Dynamic: [(3, 1, 3), (2, 6, 7)] |PASS
2. Problem 3.2 example      c=2 | Dynamic: [(1, 2, 3), (2, 6, 7)] |PASS
3. Empty                    c=1 | Dynamic: [] |PASS
4. Same Prices              c=1 | Dynamic: [] |PASS
5. All Decreasing           c=1 | Dynamic: [] |PASS
6. Single Day               c=1 | Dynamic: [] |PASS
7. One Stock, No Profit     c=2 | Dynamic: [] |PASS
8. All zeroes               c=1 | Dynamic: [] |PASS
9. Simple Profit            c=1 | Dynamic: [(1, 1, 4)] |PASS
10. Multiple Stocks          c=1 | Dynamic: [(2, 1, 2)] |PASS
11. Multiple Transactions, Different Stocks c=1 | Dynamic: [(1, 1, 2), (2, 4, 5)] |PASS
12. Large Cooldown          c=3 | Dynamic: [(1, 1, 2)] |PASS
13. Multiple Transactions with Cooldown c=1 | Dynamic: [(1, 1, 2), (1, 4, 5)] |PASS
```

Results: 13/13 tests passed

task6_unittests.py

This test file that imports dp and conducts 13 test cases covering:

- Problem 3 Examples
- No-profit / edge cases
- Profitable/Increasing Cases
- Multiple stocks
- Cooldown Cases

The output shows pass/fail per case with the result. All unit tests pass.

Description of implementation of the algorithm:

1. Step 1:
 - a. Create a list DP to store max profits up to day D
 - b. Create a list Choice to store choices made during each day
2. Step 2:
 - a. Use nested loops to iterate through the days and stocks and dynamically find the max profit for each day
3. Step 3:
 - a. Backtrack to find the sequences of transactions that lead to the max profit

Limitations of the algorithm:

- Run time could be improved
- Large matrices would not be as efficient

Comparative Analysis:

Correctness: Produces correct outputs on all unit tests including the given examples from Problem 3.

Time Complexity:

- $O(mn^2)$

Space:

- $O(n)$

Readability/Maintainability: The code uses clear variable names and provides a sufficient amount of comments to ensure readability. The code is organized for maintainability.

Trade- Off Discussion:

- Although the dynamic programming algorithm may initially seem more difficult to implement, dynamic programming helps to improve speed of the algorithm.

Analysis of the algorithms:

- Dynamic programming improves speed and reduces the redundancy that other algorithms may contain.

Lessons Learned:

- Dynamic programming is a vital part of programming due to its ability to tackle redundant tasks.
- Unit testing is important and helps us to identify potentially missed edge cases.
- Pseudocode helps to speed up the programming process and makes you think through and understand the problem first before attempting actual code.

Limitations:

- Difficult to test large matrices